

STUDY SKILLS

Making Assumptions and Critical Thinking

A Quick Guide to Understanding How to Approach Questions

A student asked me to clarify what the exam information sheet means by assumptions. The answer may help everyone, so I am sharing it with the whole class.

An assumption is a tool for handling something a question leaves open, not a way to make the question smaller. When the outcome depends on information you are not given, the most complete answer usually explains each realistic case, not just one.

An Example

In Linux, `cp` is the command that copies a file. Suppose a question asks:

What does `cp report.txt backup.txt` do?

One possible answer is: “It makes a copy of `report.txt` named `backup.txt`.” That is true, but it skips the cases that show real understanding:

- If `backup.txt` already exists, it is overwritten without a warning, and its old contents are gone.
- If `backup.txt` is the name of a directory, the file is copied into that directory instead.
- If `report.txt` does not exist, nothing is copied and an error is printed.

The first answer describes only the simplest case. The second shows how the command actually behaves.

Some Questions Are Not Like This

At the same time, some questions are not like this. They have a direct answer, and you should give it directly. Do not hunt for hidden cases in every question or pad your answers with “what if” scenarios that change nothing. That costs time, and as the information sheet says, length does not affect your score.

Approaching Questions with Critical Thinking and Analysis

This question also raises a broader point. Critical thinking and analysis, problem solving, and discuss and explain questions ask you to reason through something you may not have seen in exactly that form. That reasoning starts with recall. You cannot analyze a problem without the facts, rules, and terminology that apply to it, and correct terminology is one of the clearest ways to show what you know. When you read a question, especially one with code, try these habits:

1. **Figure out what is actually being asked.** Are you predicting what happens, explaining why, comparing ideas, or fixing something? Answering a different question than the one asked

is a common way to lose credit.

2. **Do not trust what the code looks like it should do.** Read it as the computer would, one statement at a time, and track what changes, such as variable values, the contents of a list, or which object a reference points to.
3. **Recall what applies, and use the correct terms.** Identify the concepts, rules, and details from class, the readings, or your labs that apply, and name them precisely. Explain why something happens, not just what happens.
4. **Think about what can happen in different cases.** Consider ordinary cases, edge cases such as zero or an empty collection, and exceptional situations such as invalid input or an unavailable resource. Aim to show comprehensive knowledge, but you do not need to cover every possible case, and usually you cannot. Choose the cases that best show what you understand.
5. **Question your first answer, but do not overdo it.** Look for a reason it might be wrong, and if you find one, explain it. If you studied, your first answer is probably on the right track, so checking usually means refining or adding to it, not rewriting it.
6. **Show your reasoning.** Clear reasoning can earn partial credit, and it shows me what you actually understand.

Points to Keep in Mind About Assumptions

1. **Start by answering the question that was asked.** If there is one clear outcome, explain it and move on.
2. **Explain cases only when missing information would change the result,** such as whether a file exists, which version of a class is used, or what a user types. If it would, explain each realistic case and why. If not, leave it out.
3. **Use an assumption when it does not change the outcome, or when there is only one reasonable reading.** A brief assumption is fine, and often you do not need to state it at all.
4. **An assumption is not automatically correct.** If it skips a case that changes the answer, your response may be incomplete, even if everything you wrote about your chosen case is accurate.

In Short

Assumptions can help you explain your thinking, but they do not reduce what needs to be explained. Ultimately, the goal is to show what you know. Answer each question as though you were discussing it with me, or with the person interviewing you for a job during a technical interview. You would not stop at the bare minimum or wander into things that do not matter. You would explain clearly, use the right terms, and bring up the cases that matter.